

Lecture 7 — UML Diagram to Code and Code to UML

- Goal: Understand how UML class diagrams describe program structure and how to map them to Python code.
- We'll use simplified UML boxes and examples for clear understanding.
- Includes: UML basics, examples, relationships, and practice problems with solutions.

Learning Objectives

- Understand the structure of a UML Class Diagram.
- Identify attributes, methods, and visibility in UML.
- Convert UML diagrams into Python code and vice versa.
- Recognize relationships (Association, Aggregation, Composition, Inheritance).

What is UML?

- UML = Unified Modeling Language — a visual representation of system design.
- Used to plan and communicate before coding.
- Shows classes, attributes, methods, and their relationships.
- Acts as a bridge between conceptual design and implementation.

UML Class Diagram Structure

- '-' means private, '+' means public, '#' means protected.
- Attributes define state, methods define behavior.
- Python uses naming conventions to represent visibility.

```
ClassName
-----
-attribute1: type
-attribute2: type
-----
+method1(): return_type
+method2(param: type): return_type
```

UML Visibility Mapping in Python

- + public → normal variable or method (e.g., self.name)
- - private → prefixed with `__` (e.g., self.__name)
- # protected → prefixed with `_` (e.g., self._name)
- Python uses convention instead of enforcement for access control.

Example 1 — UML to Code (Student)

- We will now implement this UML diagram into Python code.

UML Diagram:

Student

-name: str

-roll_no: int

+display(): void

Python Code for Student UML

- Attributes with '-' are made private using '__'.
- Methods with '+' are made public.
- Constructor (__init__) initializes attributes.

```
class Student:
    def __init__(self, name, roll_no):
        self.__name = name
        self.__roll_no = roll_no

    def display(self):
        print(f"Name: {self.__name}, Roll No: {self.__roll_no}")

# Example usage
s1 = Student("Ali", 12)
s1.display()
```